Teorema Master:

T(n) = a * T(n/b) + f(n)

f (n) ? n^(logb a)

1. f(n) = O(n^(logb a – eps)), eps >0
=> T(n) = Theta(n^logb a)

2. f(n) = Theta (n ^logb a)
T(n) = 2T(n/2) + n
n^logb a = n^1 = n
= > T(n) = Theta(n^logb a * log n)

2’.Master extinsa:

f(n) = Theta (n^logb a * log ^k n)

a) k > -1 => T(n) = Theta (n^logb a * log ^ (k+1) n)

T(n) = 2T(n/2) + n log n =>
T(n) = Theta(n log^2 n)

b) k = -1 => T(n) = Theta (n^logb a * log (log n))

T(n) = 2T(n/2) + n/ log n
T(n) = Theta(n log(log n))

c) k < -1 => T(n) = Theta(n^logb a)

T(n) = 2T(n/2) + n/ log^2 n
T(n) = Theta(n)

3. f(n) = W(n ^(logb a + eps)), eps >0

a*f(n/b) <= c*f(n), c apartine (0,1)

=> T(n) = Theta(f(n))

T(n) = a*T(n/b) + f(n)
a * T(n/b) = a^2 * T (n/b^2) + a * f(n/b)
…
a^k*T(n/b^k) = a^(k+1)* T(n/b^(k+1)) + a^k*f(n/b^k)

T(n) = a^(k+1) * T(1) + sum a^i * f(n/b^i), i=0:k

n/b^(k+1) = 1 =>

k+1 = logb n
=> a^(k+1) = a^(logb n) = n^(logb a)

T(n) = n^(logb a) * T(1) + sum a^i * f(n/b^i), i=0:k

daca a * f(n/b) <= c* f(n)

sum a^i*f(n/b^i) <= f(n) (1+c+c^2…) = f(n) * (1-c^(k+1))/(1-c)

Substitutie relativ la cazurile teoremei Master:

→ cazul 2: desi complexitatea este Theta(n^logb a * log n) este de preferat sa alegem Theta(n^logb a * log n + n^logb a), ca sa putem avea caz de baza n=1
→ cazul 1: desi complexitatea cu master este Theta(n^logb a), trebuie sa alegem Theta(n^logb a – f(n)) ca sa putem face calculele. In acest caz, este posibil sa avem mai multe cazuri de baza (sa nu putem cu n = 1).
→ cazul 3: se poate alege direct complexitatea Theta(f(n)), fara alte dificultati

T(n) = 9 * T(n/3) + k2*n, n> 1
k1, n = 1

a = 9
b = 3
f(n) = k2 * n

n^logb a = n^2
f(n) = O(n^(2-eps)), eps = 1

=> caz 1 Master => T(n) = Theta(n^2)

Facem substitutie considerand T(n) = Theta(n^2)

caz de baza:
c1 <= k1 <= c2 (1)

Ip. de inductie:

c1 * (n/3)^2 <= T(n/3) <= c2 * (n/3)^2 | *9 + k2*n

c1 * n^2 + k2 * n <= T(n) <= c2 * n^2 + k2 * n

vreau sa arat ca
c1 * n^2 < = T(n) <= c2 * n^2

c1 * n^2 <= c1 * n^2 + k2 * n <= T(n) <= c2 n^2 + k2 n <= c2 n^2

k2 n <= 0 , ceea ce nu se poate (k2 constanta pozitiva data)

=> aleg alta complexitate (din aceeasi clasa)

T(n) = Theta(n^2 – n)

caz de baza:
n=2:
c1 * 2 <= 9 k1 + 2 k2 <= c2 * 2
c1 <= 9/2 k1 + k2 <= c2 (1)
n=3:
c1 * 6 <= 9 k1 + 3 k2 <= c2 * 6
c1 <= 3/2 k1 + ½ k2 <= c2 (2)

Ip. de inductie:

c1 * ((n/3)^2 – n/3) <= T(n/3) <= c2 * ((n/3)^2 – n/3) | *9 + k2 * n

c1 * (n^2 – 3n) + k2 * n <= T(n) <= c2 * (n^2 – 3n) + k2 * n
c1 * (n^2 – n) + n (k2 – 2 c1) <= T(n) <= c2 * (n^2 – n) + n (k2 – 2c2)

c1 * (n^2 -n) <= c1 * (n^2 – n) + n (k2 – 2 c1) <= T(n) <= c2 * (n^2 -n) + n (k2 – 2 c2) <= c2 (n^2 - n)

k2 – 2c1 >=0, k2 -2c2 <= 0

c1 <= k2/2 <= c2 (3)

Din (1), (2) si (3) =>
c1 = min(9/2k1 + k2, 3/2k1 +1/2k2, k2/2)
c2 = max(9/2k1 + k2, 3/2k1 +1/2k2, k2/2)

T(n) = 2T(n/2) + k2 *n^3, n>1
k1, n =1

a=2, b =2

f(n) = n^3
n^logb a= n
f(n) = W (n^(1+eps)), de ex eps = 2

verificam ca

a*f(n/b) <= c*f(n), unde c apartine (0,1)

2*(n/2)^3 <= c * n^3
2*n^3/8 <= c * n^3
¼ * n^3 <=c * n^3 => putem alege c = ¼

=> din cazul 3 avem T(n) = Theta (n^3)

Facem substitutie
caz de baza:

c1 <= k1 <= c2 (1)

ip. inductie:

c1* (n/2)^3 <= T(n/2) <= c2 * (n/2)^3 | *2 + k2 * n^3
c1 * n^3 /4 + k2 * n^3 <= T(n) <= c2 * n^3/4 + k2 * n^3

c1 * n^3 + n^3 (k2 – ¾ c1) <= T(n) <= c2 * n^3 + n^3 (k2 – ¾ c2)

c1 * n^3 <= c1 * n^3 + n^3 (k2 – ¾ c1) <= T(n) <= c2 * n^3 + n^3 (k2 – ¾ c2) <= c2 * n^3

k2 – ¾ c1 >=0
k2 – ¾ c2 <= 0

c1 <= 4/3 k2 <= c2 (2)

Din (1) si (2) avem ca
c1 = min (k1, 4/3k2)
c2 = max (k1, 4/3 k2)

T(n) = 16T(n/4) + n^2

a=16, b=4, f(n) = n^2, n^logb a = n^2
caz 2 =>
T(n) = Theta(n^2 log n)

T(n) = 4 T(n/2) + n

a=4, b=2
f(n) = n
n^logb a = n^2
f(n) = O(n^(2-eps)), eps = 1
=> caz 1 => T(n) = Theta(n^2)

T(n) = 2 T(n/2) + n log n

f(n) = n log n
n^logb a = n

f(n) = W(n^(1+eps))

lim n log n / n^(1+eps) >0
n→ inf

dar lim n log n / n^(1+eps) = log n / n^eps = 0
n→inf

calculam folosind met. Iterativa:

T(n) = 2T(n/2) + n log n
2T(n/2) = 2^2 T(n/2^2) + 2 * n/2 * log n/2
…
2^kT(n/2^k) = 2^(k+1)T(n/2^(k+1)) + 2^k * n/2^k * log (n/2^k))

k+1 = log n

T(n) = n * T(1) + sum n * log (n/2^i), i=0:k

T(n) = Theta(n) + sum n * (log n – i),i=0:k

T(n) = Theta(n) + sum n log n – sum n *i, i=0:k

T(n) = Theta(n) + n * log^2 n – n * (log n – 1) * log n / 2

T(n) = Theta(n + n * log ^2 n – n * log^2 n / 2 + n * log n /2)

T(n) = Theta(n * log ^2 n /2 + n log n /2 + n)

T(n) = Theta (n log ^2 n)

T(n) = 2 * T(n-1) + 1 → turnurile din Hanoi

```
Hanoi(n, A,B,C)
{
        if (n == 0)
                return;
        Hanoi(n-1,A, C, B)
        muta(A,C)
        Hanoi(n-1,B,  A,  C)
}
```

T(n) = 2 * T(n-1) + 1
2* T(n-1) = 2^2 T(n-2) + 2
2^2*T(n-2) = 2^3 T(n-3) + 2^2
…
2^kT(n-k) = 2^(k+1) * T(n-k-1) + 2^k

n-k-1 = 1 =>
k = n – 2

T(n) = 2^(n-1) * T(1) + sum 2^i, i=0:k
= 2^(n-1) * T(1) + 2^(n-1) – 1
= Theta(2^n -1)
= Theta(2^n)

T(n) = 2T(sqrt(n)) + log n

notam n = 2^m

T(2^m) = 2T(2^(m/2)) + m

notam S(m) = T(2^m)

S(m) = 2 S(m/2) + m

=> S(m) = m log m (caz 2 master)

T(2^m) = m log m =>

T(n) = m log m =>

T(n) = log n * log (log n)

T(n) = 2T(n^1/2) + log n
2T(n/1^2) = 2^2 T(n/1^4) + log (n^(1/2)) = log n / 2

$2 * 2^{(n/2)} <= c * 2^n$